



VERIQTA

01

YAML

FOR BEGINNERS

A Practical Guide to Understanding and Writing YAML for Real DevOps Work



```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  environment: development
  port: 8080
```



Simple Explanations



Practical Examples



Real DevOps Use Cases



Common Mistakes & Fixes



Beginner Friendly



Hands-on Practice



Production Awareness



Security Best Practices



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta_academy

What Is YAML?

1. Simple Explanation

YAML is a **human-readable** data format used to write configuration files. It allows you to organize information in a clear and structured way using simple **key-value pairs**, **lists** and **indentation**.

2. Why Does It Exist?

- To provide a simple and readable **format for configuration**.
- To make it easy for humans to read **and write**.
- To organize complex data in a clear and structured way.

3. File Extensions

YAML files usually use:

- **.yaml** (most common)
- **.yml** (also widely used)

Both extensions are the same format and work in the same way.

4. Basic Example

```
name: web-app
environment: development
replicas: 3
enabled: true
port: 8080
```

5. Where Do DevOps Engineers Use YAML?



Kubernetes

Used for writing manifests to deploy and manage applications.



CI/CD

Used in GitHub Actions, GitLab CI, Jenkins and other pipelines to define build, test and deployment steps.



Docker Compose

Used to define multi-container applications such as web apps, databases and other services.



Ansible

Used to write playbooks for configuration management and automation.



Cloud Tooling

Used in tools like AWS CloudFormation, Terraform and other cloud services to define infrastructure.

6. Key Takeaway

YAML is not tied to one tool. It is widely used across DevOps, cloud and automation because it is **simple**, **readable** and **easy to work with**.

7. Junior Engineer Tip



You will see YAML everywhere in DevOps. Learning the basics will help you read, understand and modify configuration files with confidence.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta_academy

Your First YAML File

1. What Is This?

This is a simple YAML file with **key: value** pairs. Each line represents a setting. YAML is human-readable, so you can easily understand what each value means.

2. The YAML File

```
app.yaml
name: web-app
environment: development
port: 8080
```

3. Line by Line Explanation

name: web-app

→ This sets the name of the application to web-app.

environment: development

→ This sets the environment to development. It could be development, staging or production.

port: 8080

→ This sets the port number to 8080. The application will listen on this port.

4. How to Read YAML

- 1 Each line is a **key** and a **value**.
- 2 The **key** is on the **left** side.
- 3 A **colon (:)** separates the key and value.
- 4 The **value** is on the **right** side.
- 5 **Indentation** is used when you have nested data (we will learn this next).

5. Important Notes

- YAML is case-sensitive.
- Put a **space** after the colon.
- Values can be text, numbers, booleans or other data types.
- This simple format works in many DevOps tools such as Kubernetes, Docker Compose and CI/CD.

6. Common Beginner Mistakes

- ✗ Missing space after the colon (name:web-app).
- ✗ Using tabs instead of spaces.
- ✗ Writing values in the wrong format.
- ✗ Forgetting indentation when using nested data.

7. Try This



Create a file called **app.yaml** with the same content. Open it in a text editor, save it and make sure the format looks correct.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta_academy

YAML Indentation

1. What Is Indentation?

Indentation in YAML shows the **structure** and relationship between data.

It tells YAML which keys belong to which parent. Indentation is done using **spaces**, not tabs.

2. Why It Matters?

- YAML uses indentation to understand **hierarchy**.
- Incorrect indentation will cause **parsing errors**.
- Even if the content looks correct, YAML will fail if the indentation is wrong.

3. Correct Indentation (Use Spaces)

```
app:
  name: web-app
  environment: development
  server:
    port: 8080
```

Each level is indented using spaces (usually **2 spaces** or **4 spaces**). This shows that name, environment and server belong to app, and port belongs to server.

4. Incorrect Indentation (Causes Error)

```
app:
  name: web-app
  environment: development
server:
  port: 8080
```

The port key is not indented correctly. YAML will throw a **parsing error** because it does not know if port belongs to server or app.

5. Spaces vs Tabs

✓ Use Spaces

- Use 2 or 4 spaces per level.
- Spaces are consistent and work across all tools.
- This is the recommended practice.

✗ Do Not Use Tabs

- Tabs can cause parsing errors.
- Different editors treat tabs differently.
- YAML does not allow tabs for indentation.

6. Common Errors

- ✗ Using tabs instead of spaces
- ✗ Missing indentation
- ✗ Using inconsistent spacing
- ✗ Indenting too much or too little
- ✗ Mixing tabs and spaces
- ✗ Forgetting to indent nested data

7. Key Takeaway

Indentation is how YAML understands structure. Always use **spaces** (2 or 4) and keep your indentation consistent.

8. Try This



Open a text editor and create a YAML file with a nested structure. Then intentionally break the indentation and see the error.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta_academy

Strings, Numbers, Booleans & Null

1. What Are Scalar Values?

Scalar values are single pieces of data in YAML. They can be:

- Strings (text)
- Numbers (integer or decimal)
- Booleans (true or false)
- Null (no value)

These are the **basic building blocks** you will use in YAML files.

2. Example YAML File

```
app.yaml
app: nginx
replicas: 3
enabled: true
timeout: null
```

3. Explanation Line by Line

Key	Value	Type	Explanation
app	nginx	String	A text value. Here it sets the application name to nginx.
replicas	3	Number	A numeric value. Here it sets the number of replicas to 3.
enabled	true	Boolean	A boolean value. It can be true or false (without quotes).
timeout	null	Null	A null value. It means no value is set.

4. More Examples

Value	Type	Example Use
text	String	app: "web-app"
123	Number	port: 8080
true	Boolean	enabled: true
false	Boolean	debug: false
null	Null	description: null

5. Important Notes

- Strings can be written with or without quotes.
- Numbers do not use quotes.
- Booleans are true or false (lowercase).
- Null can be written as null or ~.
- YAML is case-sensitive.
- Some values like **true**, **false**, **yes**, **no**, **on**, **off** might be interpreted differently by some tools.

6. Beginner Warning



Some YAML parsers and tools may interpret certain values differently. For example, **yes**, **no**, **on**, **off**, or numbers with leading zeros can sometimes be converted to booleans or numbers unexpectedly. When in doubt, **use quotes for text** values.

7. Key Takeaway



YAML supports strings, numbers, booleans and null. These simple values are used everywhere in DevOps tools such as Kubernetes, Docker Compose, Ansible and CI/CD pipelines.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta_academy

Comments & Quoting

1. YAML Comments

- Comments start with a **# symbol**.
- Everything after # on the same line is ignored by YAML.
- Comments are used to add notes and make the file easier to understand.

2. Example with Comments

```
# Application configuration
name: web-app # name of the app
environment: development # environment
port: 8080 # application port
```

3. Quoting in YAML

- YAML supports **single quotes (')** and **double quotes (")**.
- You only need quotes in some cases.
- Quoting helps when the value might be interpreted as a **number, boolean or special character**.

4. Examples of Quoting

```
app: 'nginx' # single quotes
environment: "prod" # double quotes
version: "1.0" # looks like a number
replicas: "3" # keep as text, not number
enabled: "true" # keep as text, not boolean
path: "/var/data" # path with special char
```

5. When to Use Quotes

When to Use	Example	Why
Values that look like numbers	"8080"	To keep it as text, not a number.
Values that look like booleans	"true" or "false"	To keep it as text, not a boolean.
Values with special characters	"/var/app"	To avoid parsing issues.
Values with spaces	"my app"	Spaces can cause parsing errors.
Values with colons or symbols	"http://example.com"	To avoid misinterpretation.

6. Important Notes

- ❗ In most cases, you do not need quotes for simple text like **nginx** or **development**.
- ❗ Different YAML parsers and tools may handle some values differently.
- ❗ If a value is being interpreted incorrectly, **use quotes** to make it clear.

7. Key Takeaway

- ✓ Use # for comments.
- ✓ Use single or double quotes when needed.
- ✓ Quote values that look like numbers, booleans or contain special characters.
- ✓ Quoting prevents parsing issues and makes your YAML more reliable.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta_academy

YAML Lists

1. What Is a YAML List?

- A YAML list (also called a **sequence**) is a collection of items written using a **hyphen (-)**.
- Each item in the list is on a new line and starts with a **-**.
- Lists are commonly used in DevOps tools to define packages, ports, environments, servers and more.

2. Basic Example

```
packages.yaml

packages:
- nginx
- curl
- git
```

This defines a list of packages with three items: nginx, curl and git.

3. Other Real Examples

List of Ports

```
ports:
- 80
- 443
- 8080
```

List of Environments

```
environments:
- development
- staging
- production
```

List of Servers

```
servers:
- web-01
- web-02
- web-03
```

List of Users

```
users:
- alice
- bob
- charlie
```

4. Lists with More Details

A list can also contain dictionaries (objects) with multiple **key value** pairs.

```
containers:
- name: nginx
  image: nginx:latest
  port: 80
- name: redis
  image: redis:latest
  port: 6379
```

This is a list of two containers. Each item has its own name, image and port.

5. Important Rules

- Use a **hyphen (-)** followed by a space for each item.
- Keep consistent indentation.
- Items in a list can be simple values or complex objects.
- Lists can be nested inside other keys.
- Order matters in some tools (e.g. Kubernetes).

6. Common Mistakes

- ✗ Missing the hyphen (-).
- ✗ Incorrect indentation.
- ✗ Using tabs instead of spaces.
- ✗ Writing multiple items on the same line.
- ✗ Forgetting the space after the hyphen.

7. Key Takeaway



YAML lists are used to define multiple items in a clear and structured way. They are widely used in real DevOps configurations.

8. Try This



Create a file called **packages.yaml** with a list of at least 5 packages you want to install on a server.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta_academy

YAML Dictionaries

1. What Is a YAML Dictionary?

- A YAML dictionary (also called a **mapping**) is a collection of **key-value pairs**.
- Each **key** has a value associated with it.
- Dictionaries allow you to group related settings together.
- They are widely used in DevOps tools like Kubernetes, Docker Compose, Ansible and CI/CD.

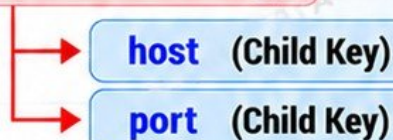
2. Basic Example

```
server.yaml

server:
  host: localhost
  port: 8080
```

3. Parent and Child Relationship

server (Parent Key)



- **server** is the parent key.
- **host** and **port** are child keys under **server**.
- Indentation shows the relationship between the parent and its children.

4. Explanation Line by Line

Line	YAML	Explanation
1	server:	This is a dictionary key (parent). It holds nested key-value pairs.
2	host: localhost	This is a child key with the value localhost.
3	port: 8080	This is another child key with the value 8080.

5. Another Real Example

```
database.yaml

database:
  host: db.example.com
  port: 5432
  user: admin
  password: secret123
```

6. Nested Dictionaries

```
app.yaml

app:
  name: web-app
  server:
    host: localhost
    port: 8080
  environment: development
```

server is a child dictionary inside the parent key **app**. It contains its own key-value pairs.

7. Common Mistakes

- ✗ Missing indentation for child keys.
- ✗ Using tabs instead of spaces.
- ✗ Placing child keys at the wrong level.
- ✗ Forgetting the colon (:) after a key.

8. Key Takeaway

- YAML dictionaries (mappings) use key-value pairs.
- Indentation shows the parent and child relationship.
- Child keys belong to the parent key above them.
- Dictionaries can be nested inside other dictionaries.
- They are commonly used in real DevOps configurations.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta_academy

Lists of Dictionaries

1. What Is a List of Dictionaries?

- A list of dictionaries is a list (**sequence**) where each item is a dictionary (**mapping**) with multiple key-value pairs.
- Each item in the list is written using a **hyphen (-)** and contains its own keys and values, indented underneath.
- This pattern is very common in DevOps tools like Kubernetes, Docker Compose, Ansible and CI/CD pipelines.

2. Basic Example

```
containers.yaml
containers:
  - name: frontend
    port: 3000
  - name: backend
    port: 8080
```

First item (dictionary)

Second item (dictionary)

3. Visual Structure

containers

```
- name: frontend
  port: 3000
```

First dictionary in the list

```
- name: backend
  port: 8080
```

Second dictionary in the list

4. Line by Line Explanation

Line	YAML	Explanation
1	<code>containers:</code>	Parent key. This is a list of dictionaries.
2	<code>- name: frontend</code>	First item in the list. name is a key with value frontend.
3	<code>port: 3000</code>	Another key in the first dictionary with value 3000.
4	<code>- name: backend</code>	Second item in the list. name is a key with value backend.
5	<code>port: 8080</code>	Another key in the second dictionary with value 8080.

5. Another Real Example

```
services.yaml
services:
  - name: web
    image: nginx
    ports:
      - "80:80"
  - name: api
    image: my-api:v1
    ports:
      - "8080:8080"
```

6. Common Use Cases

- Defining multiple containers in Kubernetes.
- Listing multiple services in Docker Compose.
- Specifying multiple environments or stages.
- Defining multiple users, servers or applications.

7. Common Mistakes

- ✗ Incorrect indentation under each list item.
- ✗ Forgetting the hyphen (-) for each item.
- ✗ Mixing up spaces and tabs.
- ✗ Placing keys at the wrong level.

8. Key Takeaway

- A list of dictionaries combines lists and key-value pairs.
- Each item in the list has its own set of keys and values.
- Indentation shows which keys belong to each item.
- This pattern is widely used in real DevOps tools.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta_academy

Nested YAML

1. What Is Nested YAML?

- Nested YAML means a key can contain **another dictionary or a list**, and this can continue to multiple levels.
- Indentation shows the **parent and child relationship** between keys.
- Nested YAML is common in tools like Kubernetes, Docker Compose, Ansible, CI/CD pipelines and cloud configuration.

2. Example of Nested YAML

```

app:
  name: web-app
  environment: production
  server:
    host: localhost
    port: 8080
  database:
    type: mysql
    host: db.example.com
    port: 3306
  
```

app (parent key)

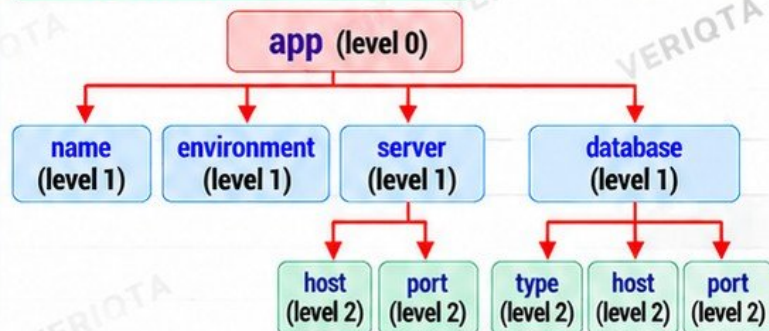
server (child keys)

database (child keys)

3. How to Read Nested YAML

- 1 Start from the left (no indentation) to find the **top level key**.
- 2 Look at the indentation to find **child keys**.
- 3 Move deeper to see more levels of nested keys.
- 4 Each indentation level represents one level in the YAML tree.

4. Visual Tree Structure



5. Line by Line Explanation

Line	YAML	Explanation
1	app:	Top level key (level 0).
2	name: web-app	Child key of app (level 1).
3	environment: production	Child key of app (level 1).
4	server:	Child key of app (level 1).
5	host: localhost	Child key of server (level 2).
6	port: 8080	Child key of server (level 2).
7	database:	Child key of app (level 1).
8	type: mysql	Child key of database (level 2).
9	host: db.example.com	Another child key of database (level 2).
10	port: 3306	Another child key of database (level 2).

6. Important Notes

- Indentation shows the structure and relationship between keys.
- Each child key belongs to the parent key above it.
- You can have multiple levels of nesting.
- Use **spaces**, **not tabs**.
- Keep indentation consistent (e.g. **2 spaces** per level).

7. Common Mistakes

- ✗ Incorrect indentation.
- ✗ Using tabs instead of spaces.
- ✗ Missing a colon (:) after a key.
- ✗ Placing a child key at the wrong level.

8. Key Takeaway



Nested YAML is a combination of dictionaries and lists. Read it like a **tree**: start from the top level and move through each indentation level to understand the structure.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta_academy

Multiline Strings

1. What Are Multiline Strings?

- Multiline strings allow you to write text that spans **multiple lines** in YAML.
- YAML provides two special syntax options for this: `|` (literal block) and `>` (folded block).
- They are commonly used for configuration files, scripts, commands, messages and documentation.

3. Example with `>` (Folded Block)

```
example2.yaml
message: >
  Deployment started.
  Waiting for application.
  This may take a few minutes.
```

- The `>` (greater than) folds multiple lines into a single line.
- Line breaks are replaced with spaces.
- Useful when you want a long sentence without manual line breaks.

6. When to Use Each?

Use Case	Use <code> </code> (Literal)	Use <code>></code> (Folded)
Preserve line breaks	✓	✗
Single line output	✗	✓
Scripts / Commands	✓	✓
Human readable messages	✓	✗
Long descriptions	✓	✓
Configuration files	✓	✓

7. Important Notes

- ! Indentation matters. The content must be indented under the key.
- ! Use `|` when line breaks are important.
- ! Use `>` when you want the text as a single line.
- ! Avoid mixing tabs. Use spaces only.

2. Example with `|` (Literal Block)

```
example1.yaml
message: |
  Deployment started.
  Waiting for application.
  This may take a few minutes.
```

- The `|` (pipe) preserves the line breaks exactly as written.
- Each line in the YAML file becomes a new line in the final string.

4. Output Comparison

| Literal Block Output

```
Deployment started.
Waiting for application.
This may take a few minutes.
```

> Folded Block Output

```
Deployment started.
Waiting for application.
This may take a few minutes.
```

5. Real World Examples

```
deployment.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  start-message: |
    Deployment started.
    Waiting for application.
    Check the logs for more details.
  command: >
    echo Deployment started.
    echo Waiting for application.
    echo Check the logs for more details.
```

8. Key Takeaway

- Use `|` to preserve line breaks.
- Use `>` to fold lines into a single line.
- Both are useful for real DevOps configurations.
- Choose based on how you want the text to appear in the final output.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta_academy

Multiple Documents

1. What Are Multiple Documents?

- YAML allows you to store multiple documents in a single file.
- Each document is separated using **three hyphens (---)**.
- This is useful when you want to define related resources together, for example in Kubernetes or other DevOps tools.

2. Document Separator (---)

- **---** indicates the **start of a new YAML document**.
- Each document after **---** is treated as a separate configuration.
- Documents in the same file are independent from each other.

3. Basic Example with Multiple Documents

```

multi.yaml

apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  environment: development
  log-level: info
--- # Start of second document
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
data:
  username: YWRtaW4=
  password: c2VjcmV0
--- # Start of third document
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: ClusterIP
  port: 80
  targetPort: 8080
  
```

Document 1 ConfigMap

Document 2 Secret

Document 3 Service

4. End of File Marker (....)

- The four dots (....) indicate the end of the last document.
- It is optional and rarely used.
- Most YAML files work fine without it.
- It can be useful when streaming or when you want to clearly show the end of a document.

5. Example with End Marker

```

example.yaml

apiVersion: v1
kind: ConfigMap
metadata:
  name: example
data:
  key: value
--- # End of file (optional)
  
```

The shows that this is the end of the last document. It is optional.

6. Why Use Multiple Documents?

- Keep related configurations in one file.
- Define multiple resources (e.g. ConfigMap, Secret, Service) together.
- Commonly used in Kubernetes and other DevOps tools.
- Makes it easier to manage and apply multiple resources at once.

7. Key Takeaway

- Use **---** to start a new YAML document.
- Use **....** to indicate the end (optional).
- Each document is separate and independent.
- Useful for defining multiple related resources together.

8. Common Mistakes

- ✗ Forgetting the **---** between documents.
- ✗ Using incorrect number of hyphens.
- ✗ Mixing indentation between documents.
- ✗ Thinking that documents share the same scope.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta_academy

YAML Anchors & Aliases

1. What Are Anchors & Aliases?

- Anchors (&) allow you to **mark a piece of content with a name**.
- Aliases (*) allow you to **reuse that content elsewhere**.
- They help you avoid repeating the same configuration multiple times.
- Very useful in DevOps configuration files like Kubernetes, Docker Compose and CI/CD pipelines.

2. Basic Example

```
config.yaml
defaults: &defaults
  retries: 3
  timeout: 30
production:
  <<: *defaults
  retries: 5
```

Anchor named "defaults" (contain key-value pairs)

Alias that reuses the "defaults" values, then overages retries

3. How It Works

- 1 Create an anchor using **&name**.
- 2 Reuse the anchored content using ***name**.
- 3 The content is copied at that location.
- 4 You can override specific values after using the alias.

4. Final Expanded Result (What It Becomes)

```
Expanded result (conceptual)
defaults:
  retries: 3
  timeout: 30
production:
  retries: 5 # overridden
  timeout: 30 # inherited
```

After using the alias and overriding **retries**, the final configuration contains all the values from defaults with the updated retries.

5. Another Real Example

```
database.yaml
common: &db
  host: db.example.com
  port: 5432
  user: app
  password: secret
dev:
  <<: *db
  database: app_dev
prod:
  <<: *db
  database: app_prod
```

Anchor with database settings

Dev environment reuses the anchor and adds its own value

Production environment reuses the anchor and adds its own value

6. Important Notes

- Anchors and aliases are part of the YAML specification.
- The merge key (<<) is a YAML feature but **not supported in every tool or implementation**.
- Some modern YAML parsers or tools may not support the merge key (<<).
- Always check what your specific tool (e.g. Kubernetes, Docker Compose, Ansible, CI/CD tool) supports.
- If merge key is not supported, you can still use anchors and aliases, but you may need to write the keys manually.

7. Use Cases

- Reusable configuration blocks.
- Common labels or annotations in Kubernetes.
- Shared environment variables.
- Default settings for different environments.

8. Common Mistakes

- ✗ Forgetting to define the anchor.
- ✗ Using an alias before it is defined.
- ✗ Expecting merge key (<<) to work in every tool.
- ✗ Overriding values without understanding precedence.

9. Key Takeaway

- Use **&** to define reusable content.
- Use ***** to reuse it.
- Merge key (<<) may not be supported everywhere.
- Always verify support in your specific tool.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta_academy

Reading YAML Like an Engineer

1. What Does It Mean?

- Reading YAML like an engineer means understanding its **structure**, not just the individual lines.
- You look at the **big picture**, then go deeper into each section.
- YAML gives you the **structure**, but the **application** using the YAML determines what the keys actually mean and how they are used.

3. Example YAML File

```

apiVersion: v1
kind: Deployment
metadata:
  name: web-app
  labels:
    app: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
  
```

app.yaml

- Top level keys (level 0)
- Child keys under metadata (level 1)
- Child keys under spec (level 1)
- Nested keys (level 2)
- List of dictionaries (level 3)
- List item (level 4)

2. The Step by Step Method

- 1 Find the **top level keys**. Start from the left (no indentation).
- 2 Follow the **indentation**. Identify parent and child relationships.
- 3 Identify **lists**. Look for items starting with **-**.
- 4 Identify **mappings**. Look for key value pairs.
- 5 Inspect the **values**. Check if they are strings, numbers, booleans, lists or dictionaries.
- 6 Understand what the **consuming tool expects**. Check the documentation for the specific tool (e.g. Kubernetes, Docker Compose, Ansible).

4. Walk Through the Example

Step	YAML	Explanation
1	apiVersion, kind, metadata, spec	Top level keys (no indentation).
2	metadata:	A child key under the top level.
3	name: web-app	A child key of metadata with value web-app.
4	spec:	Another top level key.
5	replicas: 3	A key value pair with a number.
6	template:	A child key under spec.
7	containers:	A list under spec.template.spec.
8	- name: nginx	First item in the containers list.
9	image: nginx:1.25	A key under the first list item.
10	containerPort: 80	A key with a number value under ports.

5. Important Things to Remember

- Indentation defines the structure and relationships.
- Look for top level keys first.
- Identify lists (**-**) and mappings (**key: value**).
- Inspect the values to understand the data type.
- Always check the documentation of the tool that uses the YAML to understand what the keys mean.
- The same key may mean different things in different tools.

6. Common Mistakes

- ✗ Ignoring indentation and missing the structure.
- ✗ Confusing lists with mappings.
- ✗ Assuming you know what a key means without checking the tool documentation.
- ✗ Thinking YAML alone defines the behavior.
- ✗ Skipping the documentation for the consuming tool.

7. Key Takeaway



YAML tells you the structure. The application decides what the keys mean and how they are used.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta_academy

Common YAML Errors

1. Incorrect Indentation

✗ Incorrect (will fail)

```
app:
  name: web-app
  spec:
    containers:
      - name: nginx
        image: nginx:1.25
```

✗ image is not indented under the list item.

✓ Correct

```
app:
  name: web-app
  spec:
    containers:
      - name: nginx
        image: nginx:1.25
```

✓ image is correctly indented under the list item.

2. Using Tabs Instead of Spaces

✗ Incorrect (will fail)

```
app:
→ name: web-app
```

✗ Tabs can cause parsing errors. YAML requires spaces, not tabs.

✓ Correct (use spaces)

```
app:
  name: web-app
  version: v1
```

✓ Use spaces for indentation (typically 2 spaces per level).

3. Missing Space After Colon (:)

✗ Incorrect (will fail)

```
name:web-app
port:8080
```

✗ Missing space after : can cause parsing errors or unexpected behavior.

✓ Correct

```
name: web-app
port: 8080
```

✓ Always add a space after the colon.

4. Malformed Lists

✗ Incorrect (will fail)

```
packages:
- nginx
curl
- git
```

✗ All list items must start with - .

✓ Correct

```
packages:
- nginx
- curl
- git
```

✓ Each item in a list must start with - .

5. Accidental Nesting

✗ Incorrect (will fail)

```
env:
  database:
    host: localhost
    port: 5432
```

✗ host and port are nested under database, which may not be what you intended.

✓ Correct

```
env:
  database:
    host: localhost
  port: 5432
```

✓ port is at the correct level under env.

6. Duplicate Keys

✗ Incorrect (will fail or overwrite)

```
app:
  name: web-app
  name: api
```

✗ Duplicate keys can cause parsing errors or the first value to be overwritten depending on the tool.

✓ Correct

```
app:
  name: web-app
  component: api
```

✓ Use unique keys.

7. Quoting Problems

✗ Incorrect (can cause issues)

```
port: 080
enabled: yes
version: 1.0
```

✗ Values like numbers, booleans or special formats may be misinterpreted.

✓ Correct (use quotes when needed)

```
port: "080"
enabled: "yes"
version: "1.0"
```

✓ Use quotes when the value looks like a number, boolean or special format.

8. Troubleshooting Mindset

- 1 Read the error message carefully.
- 2 Check the line number mentioned in the error.
- 3 Verify indentation and spaces.
- 4 Look for missing or extra characters (: - ' ').
- 5 Validate the file using a linter (e.g. yamllint).
- 6 Compare with a working example.
- 7 Check the documentation of the tool using the YAML file.

9. Key Takeaway

- YAML is sensitive to indentation and formatting.
- Small mistakes can cause big failures.
- Use spaces, keep structure consistent, and validate your files.
- When in doubt, check the error message and the tool's documentation.

10. Common Tools to Validate YAML

- yamllint (linter)
- kubectl apply --dry-run=client -f file.yaml (Kubernetes)
- docker compose config (Docker Compose)
- ansible-lint (Ansible)

Validate & Troubleshoot YAML

1. Why Validate YAML?

- Helps you find syntax and formatting errors before using the file.
- Prevents failures in your DevOps tools (Kubernetes, Docker Compose, Ansible, etc).
- Saves time by catching simple mistakes early.
- A valid YAML file does not always mean it is a valid configuration for a specific tool (we will cover this below).

2. Using yamllint to Validate Syntax

```
terminal
$ yamllint config.yaml
config.yaml
1:1 warning missing document start "---"
5:3 error wrong indentation: expected 2 but found 3
```

- yamllint checks the YAML syntax, indentation, formatting and common style issues.
- It does not check if the configuration is valid for a specific tool.

3. Example: Fixing a Syntax Error

Incorrect (will fail)

```
app:
name: web-app
port: 8080
env:
- prod
-dev
```

- Missing space after :
- Inconsistent indentation.
- List item not properly indented.

Correct

```
app:
  name: web-app
  port: 8080
  env:
    - prod
    - dev
```

- Proper indentation.
- Space after :
- List items correctly indented.

4. Other Useful Tools

Tool	What It Does
yamllint	Lint YAML syntax and style.
yq	Read, query and validate YAML.
pyyaml (Python)	Parse YAML and check if it is valid.
kubeval / kubeconform	Validate Kubernetes manifest files against schemas.
docker compose config	Validate Docker Compose files.
ansible-lint	Lint Ansible playbooks and YAML files.

5. Valid YAML vs Valid Configuration

Valid YAML (but may be wrong for the tool)

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers: []
```

- This is valid YAML syntax.
- But Kubernetes expects spec (not specs).
- YAML is correct, but the configuration is invalid for Kubernetes.

Valid YAML and Valid Configuration

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers: []
```

- Valid YAML syntax.
- Valid Kubernetes configuration because it uses the correct keys.
- Always check the documentation of the tool you are using.

6. Troubleshooting Steps

- Read the error message carefully.
- Check the line number mentioned in the error.
- Verify indentation and spaces.
- Make sure you are using spaces, not tabs.
- Check for missing : or incorrect list format.
- Validate the file with a linter (e.g. yamllint).
- If the YAML is valid but still fails, check the tool's documentation to confirm the correct keys and values.

8. Common Mistakes

- Ignoring error messages.
- Using tabs instead of spaces.
- YAML is valid but tool-specific keys are incorrect.
- Not checking the tool's documentation.
- Assuming a valid YAML file will always work in every tool.

7. Key Takeaway

- Validation helps you catch syntax and formatting issues.
- A file can be valid YAML but still an invalid configuration for a tool.
- Use linting tools and always check the official documentation.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta_academy

YAML in Docker Compose

1. What Is Docker Compose?

- Docker Compose uses a **YAML file** (docker-compose.yml or compose.yml) to define and run multi-container applications.
- You describe the **desired application configuration** in YAML.
- Docker Compose reads the YAML file and creates, starts and manages the containers for you.

2. Simple Example (compose.yml)

```
compose.yml

services:
  web:
    image: nginx
    ports:
      - "8080:80"
```

3. How It Works

- 1 You define **services, images, ports** and other configuration in YAML.
- 2 Docker Compose **reads** the YAML file.
- 3 It **pulls** the required images (e.g. nginx).
- 4 It **creates** and **starts** the containers as defined.
- 5 You can manage the application using **Docker Compose commands**.

4. Explanation of the Example

Line / Key	What It Means
services:	Top level key that defines the containers in your application.
web:	Name of the service (you can choose any name).
image: nginx	Tells Docker Compose to use the nginx image from Docker Hub.
ports:	Defines port mapping between your machine and the container.
- "8080:80"	Maps port 8080 on your machine to port 80 in the container.

5. Run the Example

```
Terminal

# Start the containers
docker compose up -d

# Check running containers
docker compose ps

# View logs
docker compose logs -f

# Stop and remove containers
docker compose down
```

6. What Happens

- Docker Compose pulls the nginx image (if not already available).
- Creates a container named based on the service (e.g. web).
- Maps port 8080 on your machine to port 80 inside the container.
- You can now access the Nginx web server at <http://localhost:8080>

7. YAML vs Docker Compose

YAML (The Format)	● Provides the structure (indentation, key value pairs, lists, etc.). ● Does not define what the keys mean.
Docker Compose (The Tool)	● Defines what the keys mean (e.g. services, image, ports, volumes, etc.). ● Reads the YAML file and creates the containers based on that configuration.

8. Important Notes

- ! The keys like services, image, ports are specific to Docker Compose, not to YAML itself.
- ! You can define multiple services (e.g. web, database).
- ! You can add more configuration such as environment variables, volumes and networks.

9. Key Takeaway

-  **YAML** gives you the structure.
- **Docker Compose** defines the meaning of the keys.
- You describe the desired application configuration in **YAML**, and **Docker Compose** makes it real by creating and running the containers.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta_academy

YAML in Kubernetes

1. What Is a Kubernetes Manifest?

- A Kubernetes manifest is a YAML file that describes the desired state of a resource in your cluster.
- You write the configuration in YAML and Kubernetes reads it and creates or updates the resource.
- YAML provides the **structure**, while Kubernetes defines **what the keys actually mean**.

3. Anatomy of a Kubernetes Manifest

- 1 apiVersion**
 - Specifies the Kubernetes API version for this resource.
 - Example: apps/v1
- 2 kind**
 - The type of resource to create.
 - Example: Deployment, Pod, Service.
- 3 metadata**
 - Information about the resource such as name, labels and annotations.
 - Used by Kubernetes to identify and organize the resource.
- 4 spec**
 - The desired state of the resource.
 - Structure and fields depend on the resource type (kind).
 - This tells Kubernetes what you want it to create and how it should run.

4. How Kubernetes Uses This YAML

- You apply the file using: `kubectl apply -f deployment.yaml`
- Kubernetes validates the YAML against its API schema.
- If the YAML is valid and the configuration is correct, Kubernetes creates or updates the resource.
- Kubernetes then continuously works to keep the actual state the same as the desired state defined in the YAML.

2. Example: Kubernetes Deployment

```

deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  labels:
    app: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
  
```

Annotations:

- apiVersion: apps/v1**: Kubernetes API version for this resource
- kind: Deployment**: Type of Kubernetes resource
- metadata**: Metadata about the resource (name, labels, etc.)
- spec**: Desired state (configuration specific to this resource)

5. YAML vs Kubernetes Keys

YAML (The Format)

- YAML defines the structure (indentation, key value pairs, lists, etc.).
- YAML does not define apiVersion, kind, metadata or spec.
- You could use YAML to write many different types of data.

Kubernetes (The Meaning)

- apiVersion, kind, metadata and spec are part of the Kubernetes API schema, not YAML itself.
- Their meaning is defined by Kubernetes for each resource type.
- Different resources (Pod, Deployment, Service, etc.) have different fields under spec.

6. Other Common Kubernetes Resources

- **Pod** - runs one or more containers
- **Service** - exposes Pods on the network
- **ConfigMap** - stores configuration data
- **Secret** - stores sensitive data
- **Deployment** - manages multiple Pod replicas
- **Namespace** - isolates resources

7. Common Mistakes

- ✗ Using the wrong apiVersion.
- ✗ Using an incorrect kind.
- ✗ Wrong indentation.
- ✗ Missing required fields under spec.
- ✗ Using fields that do not exist for that resource type.
- ✗ Typos in field names.

8. Key Takeaway

-  YAML gives the structure.
- Kubernetes defines the meaning of the keys.
- Always check the official Kubernetes documentation for the specific resource type and API version you are using.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta_academy

YAML in CI/CD

1. What Is YAML in CI/CD?

- CI/CD platforms use YAML files to define **automated steps** such as build, test and deployment.
- You describe the pipeline in YAML and the CI/CD platform runs the steps.
- YAML provides the **structure**, while the CI/CD platform defines **what the keys and fields actually mean**.
- Each platform (GitHub Actions, GitLab CI, Azure DevOps, etc.) has **its own YAML schema**.

3. How It Works

- You define **jobs and steps** in YAML.
- The CI/CD platform **reads the YAML**.
- It **executes the steps** in order (e.g. build → test → deploy).
- You get **automated and repeatable deployments**.

4. Common CI/CD Platforms and YAML Files

Platform	YAML File Location
GitHub Actions	.github/workflows/*.yaml
GitLab CI	.gitlab-ci.yaml
Azure DevOps	azure-pipelines.yaml
Jenkins (Pipeline)	Jenkinsfile (YAML or Groovy)
CircleCI	.circleci/config.yaml
Bitbucket Pipelines	bitbucket-pipelines.yaml

6. Important Things to Remember

- Each CI/CD platform has its own YAML schema.
- The keys and structure are specific to the platform (e.g. jobs, steps, **runs-on** in GitHub Actions).
- YAML only provides the structure. The platform decides what the keys mean and how they are executed.
- Use CI/CD to automate build, test and deployment.
- Keep your pipelines simple and readable.

2. Example: GitHub Actions Workflow

```
.github/workflows/ci-cd.yaml

name: Build Test and Deploy
on:
  push:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Set up Docker Build
        run: docker build -t my-app:latest .
  test:
    runs-on: ubuntu-latest
    needs: build
    steps:
      - name: Run tests
        run: docker run --rm my-app:latest npm test
  deploy:
    runs-on: ubuntu-latest
    needs: test
    steps:
      - name: Deploy to server
        run: echo "Deploying application..."
```

Workflow name and trigger

Build job (build the app)

Test job (run tests)

Deploy job (deploy the app)

5. What the Steps Do

Step	Explanation
Checkout code	Downloads your repository code.
Build	Builds your application (e.g. Docker image).
Test	Runs automated tests.
Deploy	Deploys to a server, cloud or cluster.

7. Common Mistakes

- Using the wrong YAML schema for the platform.
- Incorrect indentation.
- Invalid keys or fields.
- Trying to use features that are not supported by the platform.
- Hardcoding secrets (passwords, tokens, API keys) in the YAML file.
- Not checking the official documentation.

8. Key Takeaway

 **YAML in CI/CD** helps you define automated build, test and deployment steps. Always check the official documentation for your CI/CD platform to use the correct YAML schema.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta_academy

Secrets & Security in YAML

1. Why This Is Important?

- YAML files often contain configuration for applications, infrastructure and CI/CD.
- Never hardcode passwords, API keys, tokens or cloud credentials in YAML committed to Git.
- If secrets are exposed, attackers can access your systems, data and cloud accounts.
- Always use a secure method to manage and inject sensitive values.

3. Safer Approach (Use External Secrets)



Use environment variables or external secret stores instead of hardcoding values.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  database.yaml: |
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
    api_key: ${API_KEY}
```

- The actual values are stored outside the YAML file.
- The application or tool injects the values at runtime.
- This keeps your configuration safe even if committed to Git.

5. Better Options for Managing Secrets

- Environment variables (e.g. .env file, system env vars).
- Kubernetes Secrets.
- Docker secrets.
- Cloud secret managers (AWS Secrets Manager, Azure Key Vault, Google Secret Manager).
- CI/CD platform secrets (GitHub Actions, GitLab CI, Jenkins, etc.).
- External tools like HashiCorp Vault.

6. Key Takeaway



- Never commit real secrets to Git.
- Use environment variables or secret management tools.
- YAML should reference the secrets, not contain the actual values.
- Always follow the security best practices of the tool you are using.

2. Unsafe Example (Do NOT Do This)



Hardcoded credentials in YAML (using a real value)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  database.yaml: |
    username: admin
    password: MyP@ssw0rd123
    api_key: sk-abcdef123456
```



Passwords, API keys and tokens must not be stored in YAML.



Anyone with access to the repository can read them.



If the repository is public, the secrets are immediately exposed.



Even in private repositories, secrets can be leaked, copied or accidentally shared.

4. Examples for Different Tools



Kubernetes
(Secrets)

```
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
data:
  password: <base64-encoded-value>
```

Store sensitive data in a Kubernetes Secret and reference it in your Pod or Deployment.



Docker
Compose

```
services:
  web:
    image: nginx
    environment:
      - DB_PASSWORD=${DB_PASSWORD}
```

Use environment variables from a .env file or your hosting environment.



GitHub
Actions

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Use secret
        run: echo ${ secrets.DB_PASSWORD }
```

Use encrypted secrets stored in the CI/CD platform settings (e.g. GitHub Actions Secrets).

7. Common Mistakes



Hardcoding passwords, API keys or tokens in YAML.



Committing .env files with real secrets.



Using weak or easily guessable secrets.



Storing secrets in public repositories.



Not checking the official documentation for secure secret management options.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta_academy

Beginner YAML Practice Lab

1. Lab Goal

- Create a real YAML file from scratch.
- Include common fields used in real projects.
- Understand the structure and indentation.
- Break the file intentionally and observe the error.
- Fix the issue and validate the file.
- Explain the YAML structure in your own words.

2. Requirements

- 1 Application name
- 2 Environment (e.g. dev, staging, prod)
- 3 Replica count
- 4 Enabled state (true/false)
- 5 Server configuration (e.g. host, port, tls)
- 6 Port list
- 7 Container list (name and image)
- 8 Include useful comments in your YAML file

3. Example Solution (app.yaml)

```

# Application configuration
app:
  name: my-app
  environment: dev
  replicas: 3
  enabled: true

# Server configuration
server:
  host: 0.0.0.0
  port: 8080
  tls: false

# Ports exposed by the application
ports:
  - 8080
  - 9090

# Containers to run
containers:
  - name: web
    image: nginx:latest
  - name: worker
    image: my-app-worker:latest
  
```

4. Intentionally Break the Indentation

```

# app-broken.yaml
app:
  name: my-app
  environment: dev
  replicas: 3 # X wrong indentation
  enabled: true
  
```

✗ This will cause a YAML parsing error because replicas and enabled are not indented correctly under app.

5. Observe the Error

```

$ yamllint app-broken.yaml
2:1 error wrong indentation:
    expected 2 spaces
4:1 error wrong indentation:
    expected 2 spaces
  
```

i The error shows the line number and the indentation problem. YAML is sensitive to spaces and indentation.

6. Repair the File

```

# Fix the indentation and save the file
$ vim app.yaml
  
```

✓ Indent repliows and enabled correctly under app, then save the file.

7. Validate the File

```

$ yamllint app.yaml
No problems found.
  
```

✓ If you see "No problems found.", the YAML syntax is valid.

8. Explain the Structure

- app contains application details (name, environment, replicas, enabled).
- server contains host, port and tls settings.
- ports is a list of ports exposed by the application.
- containers is a list of containers with name and image.
- YAML provides the structure. The meaning of these keys depends on the tool or application using the file.

9. Your Tasks

- 1 Create the app.yaml file (use the requirements).
- 2 Inspect the file and understand the structure.
- 3 Break the indentation intentionally.
- 4 Run validation and observe the error.
- 5 Fix the file and validate again.
- 6 Explain the structure in your own words (what each section means).

10. Key Takeaway

- YAML is simple but indentation matters.
- Always validate your YAML files.
- Practice creating, breaking and fixing YAML to build confidence.
- The keys and structure depend on the application that uses the file.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta_academy